

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img = np.ones((500, 500, 3), dtype=np.uint8) * 255

# Draw a rectangle
cv2.rectangle(img, (100,100), (400,400), (0,255,0), 3)

# Draw circles
cv2.circle(img, (100,100), 8, (255,0,0), -1)
cv2.circle(img, (400,100), 8, (255,0,0), -1)
cv2.circle(img, (400,400), 8, (255,0,0), -1)
cv2.circle(img, (100,400), 8, (255,0,0), -1)

# Add text
cv2.putText(img, "DLT", (180,260),
            cv2.FONT_HERSHEY_SIMPLEX,
            2, (0,0,255), 3)

# -----
# Source Points
# -----
src = np.array([
    [100,100],
    [400,100],
    [400,400],
    [100,400]
], dtype=np.float64)

# -----
# Destination Points
# -----
dst = np.array([
    [150,80],
    [420,120],
    [380,420],
    [100,250]
])

```

```

    [0,0,0]
], dtype=np.float64)

# -----
# Direct Linear Transformation (DLT)
# -----

A = []

for i in range(4):
    x, y = src[i]
    u, v = dst[i]

    A.append([-x,-y,-1,0,0,0,u*x,u*y,u])
    A.append([0,0,0,-x,-y,-1,v*x,v*y,v])

A = np.array(A)

# Solve using Singular Value Decomposition (SVD)
U, S, VT = np.linalg.svd(A)

# Homography Matrix
H = VT[-1].reshape(3,3)

# Normalize
H = H / H[2,2]

print("Homography Matrix (DLT):")
print(H)

# -----
# Apply Transformation
# -----

warped = cv2.warpPerspective(img, H, (500,500))

# Convert BGR to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
warped_rgb = cv2.cvtColor(warped, cv2.COLOR_BGR2RGB)

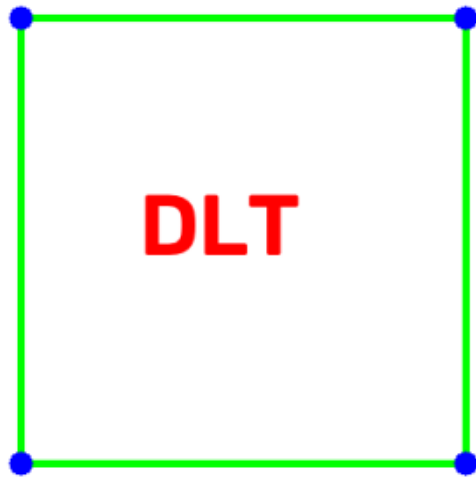
```

```
# -----  
# Display  
# -----  
plt.figure(figsize=(12,6))  
  
plt.subplot(1,2,1)  
plt.imshow(img_rgb)  
plt.title("Input Image")  
plt.axis("off")  
  
plt.subplot(1,2,2)  
plt.imshow(warped_rgb)  
plt.title("DLT Transformed Image")  
plt.axis("off")  
  
plt.show()
```

Homography Matrix (DLT):

```
[[ 7.49847716e-01 -2.47445008e-01  9.10795262e+01]  
 [ 9.75972927e-02  7.27106599e-01 -7.09983080e+00]  
 [-2.33502538e-04 -3.45177665e-04  1.00000000e+00]]
```

Input Image



DLT Transformed Image

